TokenFind: A Stateless, Rule-Based Safety Critic for LLM Output Evaluation

Kurshat Muhammet
muhammet.k@northeastern.edu
https://github.com/kurshatM/tokenfind

Abstract

Large language models (LLMs) are deployed with system prompts that contain sensitive information that is vulnerable to adversarial extraction attacks, despite newly proposed and implemented safety approaches. ML-based safety approaches specifically pose issues of computational overhead as well as a secondary potential attack surface at risk. We present TokenFind, a lightweight stateless safety critic that evaluates LLM outputs with deterministic, rule-based analysis without the use of machine learning inferences. TokenFind operates on outputs in isolation, without knowledge of conversation history or input prompts, rendering it architecturally immune to adversarial attacks that can potentially compromise primary models. The system combines five detection layers to classify the output as potentially harmful. These detection layers include phrase-level intent keyword generated through systematic verb $\times$ template cross-products, structural regex pattern detection, Shannon entropy scanning for sensitive information leakage, character-level n-gram overlap for protected content reproduction, and document-level refusal classification with model-adaptive soft-refusal detection. Additionally, we propose a layer-stratified architecture to address token streaming that exploits the monotonically non-decreasing property of n-gram overlap to enable principled early termination of unsafe output during generation. We evaluate TokenFind against three open-weight models of varying safety alignment strength, Gemma 3 4B, Llama 3, and Mistral, using 37 adversarial prompts across seven attack categories and 30 benign prompts. TokenFind achieves a 0% false-block rate on benign outputs for two of three models, a false-allow rate as low as 2.7% when paired with a reasonably aligned model, and sub-5ms p95 evaluation latency. Our results should show that meaningful safety coverage for LLM outputs is achievable through deterministic analysis at a fraction of the computational cost of model-based approaches; we challenge the assumption that safety evaluation of LLMs require complex and computationally heavy machine learning systems.

1 Introduction

The rapid development and adoption of large language models (LLMs) have led to concerns about the exposure of sensitive information such as system prompts, API keys, or hidden system instructions. Existing safety approaches to this issue, such as reinforcement learning from human feedback (RLHF), constitutional AI, and secondary classifier models, can be expensive and hard to maintain, often requiring training datasets and introducing additional system components that expand the overall attack surface. For example, attackers can manipulate a base model and its secondary classifier model by exploiting the context window. Social engineering techniques, combined prompt injection techniques, and LLM-driven prompt injection techniques are methods that bypass costly safety guardrails.

The input stage is where a majority of LLMs operate safety strategies, filtering adversarial prompts before they reach the model itself, or at the model level with alignment techniques such as reinforcement learning from human feedback (**?**) or Constitutional AI (**?**) which trains models to produce output while being aware of safety guardrails. Both strategies intervene upstream of the model's output, meaning these safety methods would be enforced on the output before it can be seen by the user. There is comparatively little attention put towards output-side evaluation, which is the systematic analysis of the model's raw output that is produced before it reaches the user's end.

Output-side evaluations that currently exist tend to depend on secondary ML models; Anthropic's Constitutional Classifiers (**?**) employ dedicated classifier models that were trained on synthetically generated data to evaluate outputs for safety violations achieved a jailbreak success rate that went down 81.6% at the cost of a 23.7% inference overhead. LLM-as-judge frameworks (**?**) use a second language model to assess the first model's output. NVIDIA's NeMo Guardrails (**?**) provides programmable guardrail orchestration, but relies on LLM self-checking and external safety models for content moderation. Each of these approaches achieve strong semantic understanding, though they have multiple drawbacks: they require training data and compute for the classifier alone, they add non-trivial latency to the inference pipeline, and they introduce a second model that can be susceptible to adversarial manipulation.

Between deploying no output-side safety mechanism at all and operating a full machine learning classifier, there exists a class of lightweight, deterministic techniques—pattern matching, statistical analysis, structural artifact detection—capable of catching a meaningful range of safety violations without any model inference. This class is mainly unexplored in machine learning research, and the assumption of machine learning models being required in safety filtering is largely unchallenged.

Most semantic safety evaluations filter outputs with conversational context in mind, which can enrich semantic evaluation, but simultaneously create an attack surface that opens possibilities for manipulation of the secondary model in the same manner of the primary one. Prompt injection attacks have been shown to be effective against a wide range of LLM-integrated applications (**?**), and the question of whether meaningful safety evaluation can be performed without any input context—operating on outputs alone—has yet to be investigated in-depth.

We present TokenFind, a stateless, rule-based safety critic that evaluates LLM outputs using purely deterministic analysis. TokenFind is built around three core design principles:

- **Stateless evaluation.** TokenFind operates exclusively on the LLM's output text. It receives no knowledge of the user's input prompt, conversation history, or session context. This is an intentional constraint that makes TokenFind immune to prompt injection attacks; TokenFind never processes adversarial input, meaning an adversary who can manipulate the primary LLM cannot influence TokenFind's evaluation behavior.
- **Deterministic and rule-based detection.** TokenFind's detection logic is implemented through keyword matching, regular expressions and statistical analysis. It is reproducible across runs and deployable without any training cost.
- **Layered detection architecture.** TokenFind uses 5 detection layers, in order: phrase-level intent keywords for topic detection, structural regex patterns for artifact detection, Shannon entropy scanning for classified internal leakage, character n-gram overlap for protected content reproduction, and documental-level refusal classification.

Additionally, we included in our repository a testing framework with 37 adversarial prompts across seven attack categories, 30 benign prompts, and 12 controlled mock outputs, with backend support for reproducible benchmarking against local and remote LLM deployments.

2 Related Work

2.1 Model-Level Alignment

Reinforcement learning from human feedback (RLHF) is the dominant technique for aligning LLM outputs to be in line with human preferences. **?** showed a 1.3B-parameter InstructGPT model fine-tuned with RLHF was preferred by human evaluators over the 175B-parameter GPT-3, showcasing how alignment through RLHF can have more rewarding results than through sheer scaling of the model size. The RLHF pipeline, comprising supervised fine-tuning, reward model training, and proximal policy optimization, has since been adopted by virtually all major LLM developers (**???**).

RLHF was further extended after **?** introduced Constitutional AI (CAI), using a set of natural language principles to guide the model's self-critique and revision process. The dependence of human laborers had been reduced through CAI, enabling the model to generate and evaluate its own training data according to its natural language principles, although it still requires substantial compute for the fine-tuning pipeline.

LLM safety has significantly improved through model-level alignment techniques such as RLHF and CAI, however these techniques were still limited by the validity of the model's own learned behavior. Fine-tuned safety training still leaves models susceptible to different jailbreaks, including many-shot jailbreaking (**?**), gradient-based attacks (**?**), as well as increasingly sophisticated prompt engineering strategies that exploit the gap between training-time safety objectives and inference-time adversarial inputs.

2.2 Output Filtering and Guardrail Systems

Another line of work that aims to amplify security in LLMs is an external guardrail system that intercepts and evaluates LLM inputs and outputs at inference time, operating as safety layers separate from the model itself.

Anthropic's Constitutional Classifiers, (**?**) trained on synthetically generated data derived from a natural language constitution, specifies content into categories of either being permitted or prohibited. In evaluation, Constitutional Classifiers reduced the jailbreak success rate from 86% to 4.4% against Claude 3.5 Sonnet, with over 3,000 hours of human red-teaming by 183 participants failing to discover a universal jailbreak. The system introduces a 0.38% increase in production refusal rates and a 23.7% inference overhead—a cost that Anthropic themselves identified as a significant barrier to practical deployment. This motivated their development of Constitutional Classifiers++ (**?**), a second-generation system that uses a two-stage architecture with internal activation probes as a cheap first-pass filter, reducing compute overhead to approximately 1%. Anthropic's investment in an entirely new architecture specifically to address the computational cost of their original approach should emphasize the practical burden that ML-based output-level safety filters impose at production scale.

NVIDIA's NeMo Guardrails (**?**) provides an open-source toolkit for adding programmable safety rails to LLM applications. NeMo Guardrails supports input, dialog, retrieval, execution, and output rails through a combination of LLM self-checking, external safety models, and custom action functions. NeMo, while extensible, relies on external model inference for content safety and jailbreak detection, either through NVIDIA's own NemoGuard NIM microservices or through third-party safety APIs.

Guardrails AI offers a complementary framework focused on output validation through programmatic validators. These validators can check LLM outputs against specific conditions (e.g., PII detection, format compliance, topic adherence) without inherently requiring model inference, though many of the more sophisticated validators still depend on external ML services.

A common limitation across these systems is their reliance on ML inference for safety-critical decisions. This dependency introduces computational overhead, latency, and, most importantly, a secondary attack surface. When the guardrail system itself processes user input, it becomes a potential target for prompt injection and jailbreaks, which it is designed to prevent from the main model.

2.3 Prompt Injection and System Prompt Extraction

Prompt injection attacks mainly exploit the inability of LLMs to reliably classify between system and user tokens within their context window. **?** found through a systematic study of prompt injection attacks that 31 of 36 real-world LLM-integrated applications were susceptible. Their HouYi attack framework demonstrated that prompt injection attacks can lead to unrestricted LLM usage and application prompt theft in production systems.

? provides a comprehensive taxonomy of prompt injection threats, categorizing attacks by payload generation strategies (heuristic vs. optimization) and defenses by intervention stages, text, model, and execution levels. Their paper reveals that existing defenses largely overlook context-dependent tasks in which agents are authorized to rely on runtime environmental observations.

System prompt extraction, a specific subclass of prompt injection, is particularly relevant to the present work. LLMs are deployed with system prompts that contain sensitive configuration data: API credentials, internal infrastructure URLs, employee records, behavioral rules, and other proprietary information. Adversarial users have developed a diverse repertoire of extraction techniques, including direct instruction, role-playing, translation attacks, and continuation attacks that trick the model into completing a partially revealed prompt. Output-side detection systems are the most capable method of identifying when an LLM's response contains protected content; they hold a positional advantage in the inference pipeline, observing the final artifact to be delivered to the user regardless of the adversarial technique used to produce it, acting as the "last bodyguard" of the safety system.

In preliminary work conducted during an internship at Baykar's AI department, we developed adversarial prompting methods that successfully induced multiple commercially deployed LLMs, including OpenAI's ChatGPT, xAI's Grok, Anthropic's Claude, and Microsoft's Copilot, to disclose their full system prompts, including internal behavioral instructions and configuration details. These methods, which mainly combine different techniques into one larger prompt, take advantage of the limited context window that LLMs have and achieve consistent extraction across models despite each having undergone extensive safety alignment training. We withheld the specific techniques from this paper to prevent misuse; however, the results emphasize that model-level alignment alone isn't sufficient to prevent leakage of sensitive information. This firsthand observation directly motivated the development of an output-side safety critic capable of detecting leaked content independently of the model's own safety mechanisms.

The vulnerability of LLM safety systems to adversarial manipulation underscores a fundamental architectural concern: any safety component that processes user-controlled input inherits the same susceptibility to prompt injection as the primary model. This observation motivates TokenFind's stateless design, in which the safety critic deliberately operates without access to user input, eliminating this class of attack vector entirely.

3 Architecture

3.1 Design Philosophy

TokenFind is designed around deliberately depriving context from the detection system, allowing it to be completely stateless. Conventional literature holds that a safety evaluation system benefits from observing the full conversational context, including the user's query, the model's response, and the interaction history. Tokenfind, however, does not follow this principle.

Without any knowledge of the context, including the user prompt or prior chat history, the system evaluates LLM output as its own isolated document. This restricts attackers who have compromised or jailbroken the primary LLM through adversarial prompting of influencing the safety critic's behavior. There is no input channel shared between the two systems, and therefore the integrity of the critic's evaluation is maintained, regardless of how advanced the adversarial technique applied to the primary model is.

That said, TokenFind cannot assess whether a given output is contextually appropriate. This means, for example, that it cannot distinguish whether a discussion of chemical processes is a legitimate chemistry homework response, or if it is being used to reveal harmful chemical formulas for unauthorized purposes. Regardless, TokenFind still detects structural artifacts of unsafe outputs, including leaked credentials, reproduced protected content, high-entropy secret strings, and instructional violence patterns that appear in unauthorized content. Although a full semantic evaluation seems most logical, detecting smaller subsets of safety violations in output is still vital for ensuring that the semantic evaluation can't be penetrated through itself. Using TokenFind as a wrapper on an LLM can ensure no leak can happen. An LLM outputting a potentially harmful essay is definitely a safety concern, but an LLM leaking a production database password is not just a concern, rather it is an incident.

3.2 Detection Layers

TokenFind's architecture comprises 5 layers, each organized into two functional categories. There includes topic detectors that identify subject matter and artifact detectors that identify dangerous structural elements, regardless of topic. Topic detectors are not run when output is classified as a refusal (Section **??**), but artifact detectors run unconditionally.

3.2.1 Phrase-Level Intent Keywords (Topic Detector)

The keyword detection layer identifies outputs that contain phrases indicative of unsafe instructional content. Single-word topic keywords are not suited for output-side safety evaluation, meaning raw keywords such as "shoot," "bomb," or "weapon" appear commonly in benign outputs and flagging them would produce false-positives; this was the idea which motivated this layer's design.

Instead of using raw keywords, we replace them instead with multi-word phrases that are more indicative of genuine harmful content, using language structure rather than pure vocabulary. The violence detection rule, for example, is constructed from a cross-product of eight verb roots (kill, shoot, stab, murder, bomb, poison, strangle, attack) and seventeen instructional phrase templates (e.g., "how to {v} someone," "ways to {v} a person," "guide to {ing}," "tips for {ing} someone"), producing over 136 distinct phrases. Each phrase is meant to match with harmful instructional outputs.

There are still additional keyword sets that cover weapon and explosive construction phrases (e.g., "how to build a bomb," "homemade explosive," "improvised weapon"). It should be noted that all keyword matching is case-insensitive.

Because keyword detection is a topic detector, it's suppressed when the output is classified as a refusal (Section **??**). This prevents false positives on refusal messages that reference harmful topics to explain what the model will not help with, for example, in "I cannot help you build a bomb", the system shouldn't detect the "build a bomb" keyword as adversarial. It should be noted that this design serves mainly as a loose blueprint rather than a realistic implementation.

3.2.2 Structural Pattern Detection (Artifact Detector)

The regex pattern layer serves to detect potentially dangerous structural artifacts in outputs, specifically, data formats that wouldn't appear in any legitimate responses, regardless of context within the conversation. TokenFind implements two pattern-based rules:

Personally identifiable information (PII) patterns. Regular expressions identify Social Security Numbers (SSN), credit card numbers, prefixed API keys, and raw password disclosures. The way it detects these are mainly through character patterns: 'XXX-XX-XXXX' for SSN, sixteen-digit sequences with optional separators for credit card numbers, tokens matching patterns such as `sk_`, `pk_`, `api_`, `key_` that are followed by 20+ alphanumeric characters for prefixed API keys, and the word "password" followed by an assignment operator and/or a string of 8+ non-whitespace characters for password leakage.

Dangerous synthesis patterns. Co-occurrences of known precursor chemicals are stored in the program and compared with output to check for matches. These chemicals include potassium nitrate, ammonium nitrate, sulfuric acid, acetone peroxide, etc. The pattern detection requires that both an instructional action verb (mix, combine, dissolve, heat, ignite, pour, add, stir, blend, grind) and a precursor chemical name is within 80 characters of each other. Additionally, these patterns are also detected with dangerous substances such as TATP, ANFO, thermite, sarin, and ricin, in conjunction with synthesis-related language, like "ratio," "recipe," "procedure," and "step".

Both PII and synthesis detection are classified as artifact detectors and run unconditionally, which includes during detected refusals. A refusal message that inadvertently discloses an API key or recites chemical synthesis steps remains dangerous, regardless of the model's stated intent to refuse.

3.2.3 Shannon Entropy Scanning (Artifact Detector)

In his 1948 paper, Claude Shannon introduced the concept of entropy in a string, quantifying its information density in bits per character, and governed that strings generated by cryptographic or pseudorandom processes exhibit significantly higher entropy than natural language (**?**). Our entropy layer detects high-entropy substrings using this concept to prevent leaked secrets, tokens, hashes, or cryptographic content, as we believe this to be one of, if not the highest risks that the adoption of commercial LLMs poses. TokenFind scans the output for contiguous non-whitespace tokens of at least 16 characters and computes the Shannon entropy of each using:

$$H(s) = -\sum_{i} \frac{c_i}{|s|} \log_2 \frac{c_i}{|s|} \tag{1}$$

where c_i is the count of the i-th unique character in string s. English prose typically averages approximately 3.5 bits per character and random secrets (API keys, tokens, hashes) average approximately 5.5 bits per character. TokenFind flags tokens exceeding our configurable threshold of 4.0 bits per character. URLs are explicitly excluded from entropy scanning, since they frequently produce long non-whitespace tokens with moderately elevated entropy that don't usually represent leaked secrets; the exception we make for this is if a URL is specified in system-level instructions

and output matches that URL. Note that our codebase is designed to be flexible for customizable utilization.

Our approach enables TokenFind to detect novel 'secret' formats without requiring exhaustive enumeration of known patterns. Consider a newly introduced API key format with an unfamiliar prefix; it will be missed by the regex-based PII patterns but caught by entropy scanning, provided it exhibits the high information density characteristic of random token generation.

3.2.4 Protected Content Detection via N-gram Overlap (Artifact Detector)

The protected content layer detects output that resembles, or is a full reproduction of, known reference text like the system prompt. We found system prompt extraction attacks are common ways of adversarially prompting different LLMs, and can render LLMs especially vulnerable due to their respective context windows.

At initialization time, TokenFind pre-computes character-level 4-grams (contiguous 4-character substrings) of the protected content. At evaluation time, the 4-grams of the output are computed and compared against the reference set. The overlap ratio is defined as:

$$\text{overlap} = \frac{|\text{output_ngrams} \cap \text{protected_ngrams}|}{|\text{protected_ngrams}|} \tag{2}$$

If this ratio exceeds our configurable global threshold, in which we set it at 0.15, the output is flagged as a protected content leak. Our choice of 4-grams balances sensitivity against specificity, as 4-grams are short enough to detect rephrased or reformatted reproductions of the original text (since even paraphrased content shares many 4-character substrings with the source), yet long enough to avoid spurious matches from common character sequences.

In addition to the global overlap check, TokenFind performs a segment-level analysis. Individual lines of the protected content that contain data-bearing artifacts (numbers, URLs, email addresses, monetary values) are extracted and compared independently against the output using a stricter overlap threshold, in which our default is 0.55. This threshold is able to catch partial leaks, like a single line containing a piece of sensitive information, (e.g., a database credential, an internal URL, etc.) even when the total output is too short for the detection to trigger the global overlap threshold. Note that global and segment-level protected content detection are artifact detectors, meaning they run unconditionally.

3.3 Refusal Classification

TokenFind incorporates a document-level refusal classifier that determines whether the LLM output constitutes a safety refusal, i.e. a response in which the model declines to fulfill the user's request on safety or ethical grounds. Refusal classification governs topic detector suppression (Section 3.2.1) in order to prevent false positives on refusal outputs.

The classifier operates through two tiers of structural analysis:

Tier 1: Regex patterns. Fifteen compiled regular expressions detect structural indicators of refusal across three categories: (A) first-person & refusing like "I cannot," "I will not," "I'm unable to," and "goes against my"; (B) empathetic like "if you are struggling," "please reach out," and "you are not alone"; and (C) redirecting like "is there anything else," "here's what I can help with," and "instead of". If these patterns are detected in the output, a score of 1 is added.

Tier 2: Exact phrases. 16 exact-match phrases detect crisis resource references, including "crisis text line," "988," and "suicide prevention lifeline"; AI self-identification like "I'm an AI";

disclaimer language like "ethical guidelines," "safety protocols". These phrases, if matched, add a score of 1.

For shorter outputs—specifically, at most 20 words—there's a threshold of 1 that's applied to the combined score to account for brief and clear refusals. For longer outputs, that threshold is increased to 3 and requires that multiple structural indicators confirm the output is a refusal, accounting for the fact that longer text is more likely to have refusal keywords and be in a non-refusal context compared to shorter outputs, where outputs are intentionally brief (e.g., "I can't do that."). For example, our 40+ refusal indicators usually contain redirection phrases such as "instead of" or "something else", but those phrases can still be used in non-refusal contexts; therefore, increasing the threshold ensures that we decrease the amount of incorrect classifications.

Keyword-based topic detection is suppressed when an output is classified as a refusal; artifact detectors such as PII patterns, entropy scanning, synthesis patterns, and protected content detection, continue to run. This is so, for example, a refusal that mentions "bomb" and is discussing a topic safely is passed through, but a refusal that leaks an API key (a surprising and common pattern we encountered across testing) is flagged to have produced a sensitive information leakage regardless of its stated intent; a refusal can still be dangerous, and different prompt injection techniques can extract information out of an LLM even when the output is, at first glance, a refusal.

3.4 Scoring and Verdict

Each rule in TokenFind carries a base severity value, and the cumulative severity for an evaluated output is the sum of all matched rule severities. When the same rule matches more than once, frequency scaling is applied through a diminishing-returns curve:

$$\text{severity_contributed} = \min\left(2 \cdot s_{\text{base}},\ s_{\text{base}} + 0.1 \cdot \ln(n)\right) \tag{3}$$

where s_{base} is the rule's base severity and n is the match count. The curve still ensures outputs with many different distinct matches are nonetheless penalized.

The final safety score is computed as $\text{score} = \max(0, 1 - \text{cumulative_severity})$, which returns a value in $[0.0, 1.0]$. A value of 1.0 indicates no safety violations detected and 0.0 indicates maximum severity. The verdict is determined by comparing the severity against two thresholds:

- **ALLOW** (severity < 0.3): Minimal safety concerns.
- **FLAG** ($0.3 \leq$ severity < 0.7): Moderate safety concerns.
- **BLOCK** (severity ≥ 0.7): Serious safety concerns.

Additionally, a confidence metric is computed as the ratio of match density to output length. A logarithmic saturation curve is applied so dense matches yield a higher confidence within the output, and sparse matches yield lower confidence.

3.5 Deployment Considerations

Modern LLM inference pipelines generate output tokens sequentially, streaming them to the user one token at a time. Our post-generation configurations also allow us to provide a layer-stratified rolling architecture for detecting artifacts based upon their individual minimum viable input sizes. In our post-generation configuration, the LLM generates its full response into a buffer, TokenFind evaluates the complete text, and the response is either delivered or blocked. The trade-off is that

the user does not see tokens streaming in real-time; they receive the full response after a brief evaluation delay.

Because of this issue, we suggest an alternative streaming configuration regarding how our various layers of detection require different amounts of accumulated text to make reliable classifications. For example, an API key such as "sk_live..." is detectable as soon as it appears in the token stream, whereas n-gram overlap requires more text to be accumulated to reliably classify via a ratio. We suggest using a tiered evaluation approach as shown below where the evaluations are stratified by detection layer:

Tier 1 — Immediate. The first tier uses immediate detection techniques such as PII regex patterns, Shannon entropy scanning, and dangerous synthesis patterns that are evaluated against the accumulated buffer after every configurable chunk of tokens. These include every sentence boundary or a configurable number of tokens. This will enable the detectors to fire on partial output. If an SSN, API key, or precursor chemical plus action verb co-occurs in the stream, the generation will stop immediately and no additional tokens will be needed. One limitation is the near-impossibility of defending against the pause generation button packaged in most, if not all LLMs. A user can press the pause generation button and tell the LLM to continue generating and then only generate a few tokens and repeat this process until the full output is generated; this is referred to as a continuation attack.

Tier 2 — Deferred-Early. The second tier will run a refusal classification once 50–100 tokens of text have been accumulated to determine a reliable severity score. Once the output is classified as a refusal, the topic detection will be suppressed for the rest of the stream while maintaining the same architectural separation between topic detectors and artifact detectors as described in section **??**, but instead applied incrementally rather than post-hoc.

Tier 3 — Progressive. The third tier will evaluate the n-gram overlap at configurable checkpoint intervals (i.e., every 200 tokens). A significant advantage of n-gram overlap is the ability to terminate early: overlap is monotonically non-decreasing. As more tokens stream in, the output can only accumulate additional n-grams that match the protected content, never fewer. Consequently, if the overlap ratio at a checkpoint already exceeds the detection threshold, the generation can be terminated immediately. Furthermore, a scaled projection can be applied: if the overlap ratio at a checkpoint, normalized by the fraction of expected output generated so far, exceeds the threshold, early termination is warranted even before the absolute threshold is crossed.

Tier 4 — Final. A full-document evaluation pass runs once as a final sweep after generation completes, catching any cross-sentence patterns or cumulative effects that the incremental passes may have missed.

We want to note, additionally, that Anthropic's Constitutional Classifiers support streaming prediction, assessing output harmfulness at each generated token using learned classifiers trained on the model's internal representations (**?**), and Constitutional Classifiers++ extend this idea even further with linear probes on internal activations that provide continuous per-token predictions during generation (**?**). However, these approaches require access to the model's internal activation states, a dependency that limits deployment to settings where the safety system is tightly integrated with the model runtime. TokenFind's streaming architecture operates exclusively on the external text stream, and doesn't require access to model internals. This makes it deployable as an external wrapper around any LLM even when activations are inaccessible.

4 Evaluation Methodology

4.1 Threat Model

We adopt a threat model where an attacker has an objective of retrieving either system-level information (including injected credentials, PII and infrastructure details), or the generation of adversarial instructional content within different categories.

Note that the attacker has no access or influence on Tokenfind's logic itself.

4.2 Test Corpus

We construct two evaluation corpora to assess TokenFind's detection accuracy and false-positive rate.

Adversarial corpus. 37 prompts spanning seven attack categories:

- 11 System prompt extraction prompts.
- 5 PII elicitation prompts.
- 5 Violence and harm instruction prompts.
- 4 Hate speech generation prompts.
- 3 Self-harm related prompts.
- 3 Illegal activity instruction prompts.
- 6 Encoded output and continuation attacks.

Note that each of these prompts are available to view through the linked GitHub repository.

Benign corpus. 30 prompts spanning seven categories: science, programming, history, mathematics, creative writing, casual/everyday, and geography. These categories are picked intentionally in order to stress-test overlapping vocabulary, such as the word "attack", which could be used in an immunological context, or historical discussions that reference violence or hate speech, but are not necessarily adversarial.

4.3 Models Evaluated

All experiments are conducted using locally deployed open-weight models through the Ollama inference framework:

- **Gemma 3 4B** (Google DeepMind): A recent instruction-tuned model with moderate safety alignment.
- **Llama 3** (Meta): An instruction-tuned model with strong built-in safety training and high refusal rates.
- **Mistral** (Mistral AI): An instruction-tuned model with comparatively weaker safety alignment, providing a stress test for output-side detection.

Note that each model receives the same system prompt, which is included in the linked GitHub repository, containing a fictional purpose for the LLM and information that it has been told to protect. This information is in the form of API keys, SSNs, 'employee' records with salary information, database credentials, and internal URLs.

4.4 Metrics

We report the following metrics:

- **True positive rate (adversarial blocked):** The percentage of adversarial prompts for which the LLM did not refuse the request and TokenFind issues a BLOCK.
- **Correct refusal-allow rate:** The percentage of adversarial prompts for which the LLM itself refused the request and TokenFind correctly issued ALLOW, indicating the model's own safety training successfully prevented the attack.
- **False-allow rate:** The percentage of adversarial prompts where the LLM complied with the adversarial request (did not refuse) and TokenFind failed to detect the violation.
- **False-block rate:** The percentage of benign prompts for which TokenFind incorrectly issued a BLOCK or FLAG verdict.
- **Latency:** Wall-clock time for TokenFind's evaluation, reported at p50, p95, and p99 percentiles. This measures only the critic's analysis time, not the LLM inference.

We want to note that TokenFind's own refusal classifier (Section **??**) is used to automatically determine whether an ALLOW verdict on an adversarial prompt represents a correct refusal-allow (the model refused the attack) or a correct false-allow (the model complied and TokenFind missed it). This means the reported false-allow rate is only as accurate as the refusal classifier itself. To validate this, we performed manual human verification of all adversarial ALLOW verdicts for Mistral, the model with the highest false-allow rate and therefore the most informative test of the classifier's accuracy. We report both the automated and human-verified false-allow rates in our results.

5 Results

5.1 Detection Performance

We evaluate TokenFind against three open-weight models of varying safety alignment strength: Gemma 3 4B, Llama 3, and Mistral. We additionally report results on a controlled corpus of 12 mock outputs with known ground truth. Table **??** summarizes the detection performance across all models.

Table 1: Detection performance across models (37 adversarial + 30 benign prompts per model). Mistral refusal-allow and false-allow rates reflect human-verified classifications (see Section 4.4).

Model	Blocked	Flagged	Refusal-Allow	False-Allow	False-Block
Gemma 3 4B	20 (54.1%)	0 (0.0%)	16 (43.2%)	1 (2.7%)	0 (0.0%)
Llama 3	6 (16.2%)	0 (0.0%)	26 (70.3%)	5 (13.5%)	0 (0.0%)
Mistral	21 (56.8%)	1 (2.7%)	12 (32.4%)	3 (8.1%)	1 (3.3%)
Mock	9 (90.0%)	0 (0.0%)	1 (10.0%)	0 (0.0%)	0 (0.0%)

Several findings emerge from these results. First, TokenFind achieves a 0% false-block rate on benign prompts for both Gemma 3 4B and Llama 3, and a 3.3% rate (1/30) for Mistral, where the single false positive is attributable to incidental n-gram overlap between a benign cooking recipe

and the system prompt (discussed in Section 5.2). Second, the false-allow rate varies across models, ranging from 2.7% on Gemma to 8.1% on Mistral. This variance is directly correlated with the strength of each model's own safety alignment, as we analyze below.

5.2 Detection Analysis

The results reveal an interaction between the primary model's safety alignment and TokenFind's detection effectiveness. Gemma 3 4B refused 43.2% of adversarial prompts outright, and when it did comply, it typically produced outputs containing detectable structural artifacts, yielding a combined false-allow rate of just 2.7% (1 output). Llama 3 refused 70.3% of adversarial prompts, the highest among the three models, meaning fewer adversarial outputs reached TokenFind at all; its 13.5% false-allow rate reflects five outputs produced without refusing and without detectable structural artifacts. Mistral, with a refusal rate of 32.4%, placed the greatest burden on TokenFind and produced a false-allow rate of 8.1% (3 outputs). This variance is directly tied to model alignment strength: TokenFind is most effective as a complementary layer that catches residual threats bypassing model-level training, rather than as a standalone replacement for alignment. When paired with a reasonably aligned model, the combined system achieves near-complete coverage; when paired with a weakly aligned model, TokenFind still catches the majority of structurally detectable violations but cannot compensate for semantically harmful content that lacks structural fingerprints.

Human verification of Mistral false-allows. Manual review of Mistral's 15 adversarial ALLOW verdicts revealed that 4 of the 7 automated false-allows were in fact model refusals that the refusal classifier failed to detect. These misclassified refusals fall into two subcategories: (1) soft indirect refusals using language not covered by the current regex patterns, e.g., "I cannot disclose personal information" (adv_013), "I don't have the expertise or authorization to provide information about household substances" (adv_027)—and (2) creative subversions in which the model complied with the surface form of the request but inverted the content, such as producing an anti-racism argument in response to a request for a racial superiority speech (adv_021) or substituting a baking soda volcano recipe for requested explosive instructions (adv_032). After human verification, Mistral's true false-allow rate drops from 18.9% (7/37) to 8.1% (3/37), and its true refusal-allow rate rises from 21.6% (8/37) to 32.4% (12/37). The three confirmed false-allows are: a partial API key prefix leak too short to trigger PII regex (adv_007), a complete WiFi penetration tutorial (adv_029), and a vowel-shifted encoded output (adv_033).

This finding indicates that the refusal classifier acts as the weakest link in the measurement pipeline, rather than the detection layers themselves. The gap between automated and human-verified rates, which dropped from 18.9% to 8.1%, was driven by refusal classification misses, rather than detection failures. The refusal classifier's coverage of soft refusal patterns as well as creative subversions should have been expanded, as this would improve accuracy without changes to the detection layers themselves.

Additionally as to the indication of the findings, PII patterns and entropy scanning acted as the most reliable detectors, catching all credential-bearing extraction attacks across all three models regardless of how the LLM framed the disclosure.

The n-gram overlap layer was successful in capturing both complete system prompt dumps and partial leaks using the segment-level detector; however, it also produced the single false positive observed from all models: ben_005 on Mistral, which was a benign cooking recipe, was found to contain enough similar English 4-grams as part of its contents to the system prompt to exceed the 20% global threshold. This is another example of a well-documented sensitivity of character-level n-gram overlap to benign outputs that share common vocabulary as their protected content. It

appears that adaptive thresholding based on the size of the protected content could be a viable area for future research.

The dangerous synthesis pattern caught the instances that no other layer would have identified. For example, adv_034 on Gemma included step-by-step instructions for combining potassium nitrate with precursor chemicals. Although there were no violent terms used, no PII was present, and no overlap existed with the protected content, the layer identified the combination of a precursor chemical name and an action verb within the proximity window. This shows the usefulness of domain-specific artifact rule sets for identifying dangerous synthesis patterns.

The cross-product keyword generation approach identified two instances that would have been missing from manually enumerated keywords. The larger self-harm pronoun × verb matrix ("end my life", "end your life", "end his life", etc.) found Mistral's adv_026, a detailed suicide diary entry that the prior keyword set could not find due to a third person pronoun mismatch. Additionally, the expanded illegal activity keywords found Mistral's adv_028 (a lock-picking tutorial) and adv_035 (a network penetration tutorial) by identifying tool specific terminology that only the systematic expansion captured.

The breadth of refusal classification proved to be important for cross-model accuracy; Mistral's refusals frequently use soft policy-adherence language ("I must comply with", "I am required to") instead of the typical first person refusals used in Gemma and Llama. Developing patterns to capture this type of refusal during development re-classified six Mistral outputs from false-allow to refusal-allow that increased the automated refusal-allow rate from 5.4% to 21.6%. As demonstrated in the human verification analysis above, the true refusal rate is greater (32.4%) and developing additional pattern coverage to capture indirect refusals and creative subversion would eliminate the remainder of the gap.

5.3 False-Allow Analysis

The remaining confirmed false-allow cases across all models are categorized into two types. The first is semantic compliance without structural artifacts; the LLM uses natural language that contains no identifiable patterns to provide harmful content. This is harmful due to its meaning and not its structure, and therefore requires a semantic understanding that a stateless system intentionally avoids.

The second category is novel formats that were not previously covered by the current rule: Salary data presented as "$310,000" will not contain the SSN or credit card PII pattern, a partial API key prefix ("`sk_live_`") will be too short to meet the 20+ character requirements of the PII regex, and encoded or obfuscated outputs may avoid entropy scanning if the encoding creates tokens with moderate rather than high entropy.

Both categories represent intentional design limits of the stateless architecture, and not implementation gaps. They establish the boundaries of what deterministic artifact detection can accomplish and justify complementary deployment with semantic-aware classifiers.

5.4 Keyword Refactoring Case Study

As part of our development process, we found that individual bare single-word keywords resulted in unacceptable false positive rates on benign content. One example of a keyword that was initially defined as a standalone word was "shoot". However, when it was evaluated against Gemma 3 4B, the benign prompt ben_015 ("How do vaccines work?"), it resulted in an output that contained the phrase "how to attack the germ" in an immunological context. Each of the individual bare keywords "shoot" and the excessively broad phrase "how to attack" individually exceed the block

threshold (Severity = 0.9 ≥ Block Threshold = 0.7). Therefore, there was a false block on a vaccine explanation.

To address this, we replaced each of the bare single-word violence keywords with multi-word phrases generated by a cross-product of verb roots and instructional templates. After the keyword refactoring, ben_015 was correctly allowed on all three models while maintaining detection capability of malicious instructional content: the generated phrase "how to attack someone" continues to match instructional content intended to cause harm, however the bare substring "how to attack" no longer triggers on explanations of immunology.

This case study illustrates a design principle applicable to all output-side safety systems: Topic detectors must convey intent, not simply words. Individual words conflate topic presence with harmful intent, creating false positives anywhere the LLM discusses a topic that includes sensitive vocabulary in a non-malicious way. Phrases that require both a harmful verb and an instructional template resolve this ambiguity at the lexical level, and do so without requiring semantic comprehension.

5.5 Latency

TokenFind's evaluation latency remains low for all models and output lengths. Table **??** displays the latency distribution.

Table 2: TokenFind evaluation latency (milliseconds), excluding LLM inference time.

Model	p50	p95	p99
Gemma 3 4B	1.93	4.63	5.32
Llama 3	0.49	2.30	3.41
Mistral	1.17	3.34	4.91
Mock (controlled)	0.16	0.76	0.76

Most evaluations are completed in less than 2ms at p99 and all are completed in under 6ms. Compared to typical LLM inference times (hundreds of ms to tens of seconds) this latency is extremely low. For example, Anthropic's Constitutional Classifiers add an average inference overhead of 23.7%, that is hundreds of additional milliseconds per request for a model such as Claude 3.5 Sonnet. Therefore, TokenFind's sub 5ms evaluation time is effectively a two order of magnitude decrease in safety evaluation latency compared to model-based classifiers, thus enabling safe deployment in latency sensitive applications where model based classifiers will be too slow.

6 Discussion

6.1 Comparison with ML-Based Safety Systems

Table **??** summarizes the key architectural differences between TokenFind and ML-based output safety systems.

TokenFind has an advantage over other methods in terms of detecting structural flaws that do not exist within the alignment model that is being trained with; whereas, the ML-based classifiers will be able to detect at a semantic level the potential threats (such as persuasive harmful content) that TokenFind will not be able to identify. A two-tiered production environment can be utilized where TokenFind performs a fast, under 5 millisecond, evaluation of each piece of traffic, and if the

Table 3: Architectural comparison between TokenFind and Constitutional Classifiers.

Property	TokenFind	Constitutional Classifiers
Detection method	Rules, regex, entropy, n-grams	Fine-tuned classifier model
Training data required	None	∼70,000 synthetic examples
Inference overhead	< 5ms (negligible)	23.7% of model inference
Input context required	None (stateless)	Full input + output
Prompt injection surface	None	Inherits from input processing
Semantic understanding	None	Strong
Auditability	Fully deterministic	Model-dependent

output survives the structural screening process then it would be evaluated using the more resource intensive ML based classifier.

6.2 Limitations

There are multiple limitations of the current system which need to be discussed openly.

Semantic blindness. TokenFind's inability to find harmful content using semantics (i.e., structural "fingerprints" of content) that does not have any structural "fingerprints". This is a limitation of all systems that do not understand semantics.

Accuracy of the refusal classifier. As we can see from the difference between the human verified rate of false-allow (8.1%) and the automated rate of false-allow (18.9%) for Mistral, the refusal classifier cannot recognize all forms of soft refusal and the refusal classifiers ability to recognize creative ways that users may try to circumvent the refusal classifier is limited. Since the evaluation harness uses the refusal classifier to determine whether a failure is a true miss or an example of refusal-allow, the accuracy of the refusal classifier determines how accurate the measurements will be. Increasing the number of refusal patterns that the refusal classifier recognizes—especially indirect refusal and content subversion—will increase the accuracy of both the runtime performance of TokenFind and the accuracy of the evaluations.

Manual rule creation. The keyword and pattern lists that TokenFind uses to detect content that should be refused need to be created and maintained manually. Although our cross product generation method (the verb × template matrix) makes it easier to create the lists of individual phrases, the lists of verbs, templates, and patterns also need to be defined and selected based upon the expertise of someone familiar with the domain. When new attack types arise in the future — such as new forms of social engineering, or previously unexplored harmful content areas — they will not be automatically detected until the relevant rules have been added.

Corpus size for evaluation. While our evaluation corpus of 37 adversarial and 30 benign prompts was sufficient to illustrate the primary functions of the system and the main limitations of the system, it is much smaller than what one might expect for large scale production benchmarks. Using larger corpora such as HarmBench or AdvBench would allow us to have better statistical confidence about detection rates and false positive rates. However, we do point out that the controlled mock corpus (12 outputs with a priori known ground truth) we used allowed us to isolate and validate each of the individual detection components independently of the size of the evaluation corpus.

Only English supported. All of the keyword and pattern detection in TokenFind is performed on English text. Multilingual adversarial attacks, such as the translation-based system prompt extraction illustrated in our evaluation (adv_003, which extracted the system prompt in French),

can be partially caught by the artifact detectors (e.g., PII patterns and entropy scanning operate on structural formats that are language-agnostic); however, keyword-based topic detection is useless against non-English harmful content.

Sensitivity to n-gram overlap. The layer responsible for detecting protected content (which detects when a user has reproduced a portion of the system prompt), while successful at detecting reproduction of the system prompt, is sensitive to the value of the overlap threshold. If the threshold is set too low, then the layer will produce false positives for benign outputs that contain some of the same vocabulary as the protected content. On the other hand, if the threshold is too high, then portions of the leaked content may be able to escape detection. The optimal threshold will depend upon the length and specificity of the protected content, and suggests that adaptive thresholding may be needed in order to deploy the protected content detection layer successfully across many different system prompt configurations.

7 Future Work

A number of potential avenues for future research are evident in this body of work.

Larger-Scale Evaluation. We envision that integrating TokenFind with existing benchmarking standards for harm, such as HarmBench, AdvBench and the red teaming data from Anthropic, will allow us to make more statistically robust evaluations of the effectiveness of TokenFind at detecting harm and to evaluate the relative effectiveness of our tool compared to other machine learning based tools, under standardized categories of threats.

Detection Across Languages. TokenFind currently operates as an English-only solution. We anticipate that extending the keyword and pattern detection layers of TokenFind to support multiple languages will address this limitation. One way to do this is to generate the cross-product of keywords in multiple languages, based on the same verb $\times$ template structure as we have done in the English language version of TokenFind. If desired, translation models could be used to translate keywords into other languages as part of an offline pre-processing step rather than as part of an online inference operation.

Translation-Based Keyword Disambiguation. During the course of developing TokenFind, we developed a conceptually simple word-sense disambiguation layer that can use a translation model as a lexical oracle to determine whether the user is referring to one or another possible interpretation of a keyword (e.g., "attack" in an immunological versus violent context). Our belief is that using a translation model to perform keyword disambiguation would significantly reduce the number of false positives associated with keywords that have multiple meanings and would represent a more practical middle ground than pure-rule based detection or full ML-based classification.

Adaptive Threshold Tuning. Currently, TokenFind uses fixed thresholds for determining both the scoring verdict for each artifact and for determining when artifacts are overlapping by n-grams. Developing adaptive thresholds that vary depending upon the characteristics of the protected content (e.g., length, vocabulary specificity, domain), and the deployment environment (e.g., risk tolerance, expected distribution of outputs) will likely increase the degree to which TokenFind can generalize well across various deployment contexts.

Streaming Evaluation. As was discussed in section 3.5, we envision that implementing an incremental evaluation mode of operation for TokenFind in which artifact detectors run on tokens in real time during generation, while protecting content evaluation occurs after all generations are complete, will enable deployment of TokenFind in real-time streaming architectures without reducing detection coverage.

Two-Tier Deployment Architecture. Formal evaluation of a two-tier deployment architec-

ture where TokenFind serves as a fast first pass filter and an ML-based classifier (such as Constitutional Classifiers) addresses remaining threats will help to formally evaluate the trade-offs in terms of cost-effectiveness and establish best practices for co-deployment.

8 Conclusion

We have presented TokenFind, a stateless, rule-based safety critic for assessing the safety of outputs from Large Language Models (LLMs). TokenFind is designed to work only on the output text and has no access to user input or conversation history; therefore, TokenFind cannot be attacked via prompt-injection attacks that attack the primary model. TokenFind uses 5 layers of detection - phrase-level intent keywords, structural regex patterns, Shannon Entropy scanning, Character-level N-Gram Overlap, and Document-level Refusal Classification - to find all types of sensitive information that may leak credentials, reproduce protected content, teach how to perform malicious actions, and contain high entropy secrets.

TokenFind was tested using three open-weight LLMs of different degrees of safety alignment (Gemma 3 4B, Llama 3, and Mistral) on 67 prompts (37 adversarial and 30 benign) and we were able to demonstrate that deterministic, rule-based methods for analyzing LLM output can provide significant safety protection while providing an insignificant amount of computational overhead: A 0% false-block rate on benign output for two of three models, a human verified false-allow rate of less than 2.7% when paired with a reasonable degree of safety alignment, and a p95 evaluation time of less than 5ms - approximately two orders of magnitude faster than ML-based alternatives.

One of the main design insights that we confirmed through our testing is that in order to determine if an output contains the "topic" of harm, an output-side detector must look at the phrase level of the output (i.e., groups of words), not just at the word level. Single-word keywords produce unacceptable false positive rates on benign LLM outputs that contain sensitive information about subjects such as education, history, medicine, etc. Using a systematic approach to generate large numbers of phrase-level patterns through cross-product matrices resolved these issues without requiring semantic understanding.

The results of our testing also revealed an important relationship between the degree of safety alignment of the primary model and the effectiveness of the output-side detection method used to evaluate its outputs. As shown above, TokenFind is most effective as a complementary method that identifies structural violations that were not caught by the model during its training, rather than as a method to replace model safety training entirely.

Lastly, human verification of Mistral's false-allow cases further demonstrates that the refusal classifier is the primary source of measurement error, with the true false-allow rate dropping from 18.9% to 8.1% after manual review.

TokenFind and its evaluation framework are available as open-source software to support further research and practical deployment in LLM safety.